TASK 28:ASSIGNMENT NO. 28

Name of Student : Anuja Vilas Wankhade

Domain : Artificial intelligence and machine Learning (AIML)

College Name: Manav School of Polytechnic AKOLA

Branch : Computer Engineering

GitHub Repository: https://github.com/anujaw193-star/python-project-collection

---

1. (Loading the Dataset)

   Code:

```
[1]
✓ 0s
import pandas as pd
import numpy as np
import zipfile
```

```
[3]
✓ 0s
with zipfile.ZipFile("/content/train.txt.zip", "r") as zip_ref:
  zip_ref.extractall("/content/")
df = pd.read_csv(
      "/content/train.txt",
      sep=";",
      names=["text", "emotions"]
  )

print("First 10 Rows")
print(df.head(10))

print(df.shape)

print(df.isnull().sum())
```

   Output:

```
First 10 Rows
                                                text  emotions
0                            i didnt feel humiliated   sadness
1  i can go from feeling so hopeless to so damned...   sadness
2   im grabbing a minute to post i feel greedy wrong     anger
3  i am ever feeling nostalgic about the fireplac...      love
4                                i am feeling grouchy     anger
5  ive been feeling a little burdened lately wasn...   sadness
6  ive been taking or milligrams or times recomme...  surprise
7  i feel as confused about life as a teenager or...      fear
8  i have been with petronas for years i feel tha...       joy
9                                i feel romantic too      love
(16000, 2)
text        0
emotions    0
dtype: int64
```

---

2. (Exploring Target Labels)

   Code:

```
[5]
✓ 0s
from sklearn.preprocessing import LabelEncoder

print("Unique Emotion Labels:")
print(df["emotions"].unique())

label_encoder = LabelEncoder()
df["emotion_encoded"] = label_encoder.fit_transform(df["emotions"])

label_mapping = dict(zip(label_encoder.classes_, label_encoder.transform(label_encoder.classes_)))
print("\nLabel Mapping:")
print(label_mapping)

print("\nFirst 10 Rows with Encoded Labels:")
print(df.head(10))
```

Output:

```
Unique Emotion Labels:
['sadness' 'anger' 'love' 'surprise' 'fear' 'joy']

Label Mapping:
{'anger': np.int64(0), 'fear': np.int64(1), 'joy': np.int64(2), 'love': np.int64(3), 'sadness': np.int64(4), 'surprise': np.int64(5)}

First 10 Rows with Encoded Labels:
                                                text  emotions  \
0                            i didnt feel humiliated   sadness
1  i can go from feeling so hopeless to so damned...   sadness
2   im grabbing a minute to post i feel greedy wrong     anger
3  i am ever feeling nostalgic about the fireplac...      love
4                              i am feeling grouchy     anger
5  ive been feeling a little burdened lately wasn...   sadness
6  ive been taking or milligrams or times recomme...  surprise
7  i feel as confused about life as a teenager or...      fear
8  i have been with petronas for years i feel tha...       joy
9                               i feel romantic too      love

   emotion_encoded
0                4
1                4
2                0
3                3
4                0
5                4
6                5
7                1
8                2
9                3
```

---

3. (Lowercasing)

Code:

```
[6]
✓ 0s
df["original_text"] = df["text"]

df["text"] = df["text"].str.lower()

print("Before and After Lowercasing:\n")

for i in range(5):
  print(f"Sample {i+1}")
  print("Before :", df["original_text"][i])
  print("After :", df["text"][i])
  print("-" * 60)
```

Output:

```
Before and After Lowercasing:

Sample 1
Before : i didnt feel humiliated
After : i didnt feel humiliated
------------------------------------------------------------
Sample 2
Before : i can go from feeling so hopeless to so damned hopeful just from being around someone who cares and is awake
After : i can go from feeling so hopeless to so damned hopeful just from being around someone who cares and is awake
------------------------------------------------------------
Sample 3
Before : im grabbing a minute to post i feel greedy wrong
After : im grabbing a minute to post i feel greedy wrong
------------------------------------------------------------
Sample 4
Before : i am ever feeling nostalgic about the fireplace i will know that it is still on the property
After : i am ever feeling nostalgic about the fireplace i will know that it is still on the property
------------------------------------------------------------
Sample 5
Before : i am feeling grouchy
After : i am feeling grouchy
------------------------------------------------------------
```

4. (Removing Punctuation)

Code:

```
[8]
✓ 0s
import string
df["before_punctuation"] = df["text"]
def remove_punctuation(text):
  return text.translate(str.maketrans("", "", string.punctuation))

df["text"] = df["text"].apply(remove_punctuation)

print("Before and After Removing Punctuation:\n")

for i in range(5):
  print(f"Sample {i+1}")
  print("Before :", df["before_punctuation"].iloc[i])
  print("After :", df["text"].iloc[i])
  print("-" * 60)
```

Output:

```
Before and After Removing Punctuation:

Sample 1
Before : i didnt feel humiliated
After : i didnt feel humiliated
------------------------------------------------------------
Sample 2
Before : i can go from feeling so hopeless to so damned hopeful just from being around someone who cares and is awake
After : i can go from feeling so hopeless to so damned hopeful just from being around someone who cares and is awake
------------------------------------------------------------
Sample 3
Before : im grabbing a minute to post i feel greedy wrong
After : im grabbing a minute to post i feel greedy wrong
------------------------------------------------------------
Sample 4
Before : i am ever feeling nostalgic about the fireplace i will know that it is still on the property
After : i am ever feeling nostalgic about the fireplace i will know that it is still on the property
------------------------------------------------------------
Sample 5
Before : i am feeling grouchy
After : i am feeling grouchy
------------------------------------------------------------
```

5. (Removing Numbers)

Code:

```
[16]
✓ 0s
```

```python
import re
df["before_numbers"] = df["text"]

def remove_numbers(text):
  return re.sub(r'\d+', '', text)
df["test"] = df["text"].apply(remove_numbers)
print("Before and After Removing Numbers:\n")

for i in range(5):
    print(f"Sample {i+1}")
    print("Before :", df["before_numbers"].iloc[i])
    print("After :", df["text"].iloc[i])
    print("-" * 60)
```

Output:

```
Before and After Removing Numbers:

Sample 1
Before : i didnt feel humiliated
After : i didnt feel humiliated
------------------------------------------------------------
Sample 2
Before : i can go from feeling so hopeless to so damned hopeful just from being around someone who cares and is awake
After : i can go from feeling so hopeless to so damned hopeful just from being around someone who cares and is awake
------------------------------------------------------------
Sample 3
Before : im grabbing a minute to post i feel greedy wrong
After : im grabbing a minute to post i feel greedy wrong
------------------------------------------------------------
Sample 4
Before : i am ever feeling nostalgic about the fireplace i will know that it is still on the property
After : i am ever feeling nostalgic about the fireplace i will know that it is still on the property
------------------------------------------------------------
Sample 5
Before : i am feeling grouchy
After : i am feeling grouchy
------------------------------------------------------------
```

6. (Removing Emojis & Special Characters)

Code:

```
[17]
✓ 0s
```

```python
df["before_ascii"] = df["text"]
def remove_emojis_special(text):
  return text.encode('ascii', 'ignore').decode('ascii')

df["text"] = df["text"].apply(remove_emojis_special)

print("Before and After Removing Emojis and Special Characters:\n")

for i in range(5):
    print(f"Sample {i+1}")
    print("Before :", df["before_ascii"].iloc[i])
    print("After :", df["text"].iloc[i])
    print("-" * 60)
```

Output:

```
Before and After Removing Emojis and Special Characters:

Sample 1
Before : i didnt feel humiliated
After : i didnt feel humiliated
------------------------------------------------------------
Sample 2
Before : i can go from feeling so hopeless to so damned hopeful just from being around someone who cares and is awake
After : i can go from feeling so hopeless to so damned hopeful just from being around someone who cares and is awake
------------------------------------------------------------
Sample 3
Before : im grabbing a minute to post i feel greedy wrong
After : im grabbing a minute to post i feel greedy wrong
------------------------------------------------------------
Sample 4
Before : i am ever feeling nostalgic about the fireplace i will know that it is still on the property
After : i am ever feeling nostalgic about the fireplace i will know that it is still on the property
------------------------------------------------------------
Sample 5
Before : i am feeling grouchy
After : i am feeling grouchy
------------------------------------------------------------
```

7. (Removing Stopwords)
   Code & Output:

```
18]
✓ 0s
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

nltk.download('punkt')
nltk.download('stopwords')

stop_words = set(stopwords.words('english'))
df["before_stopwords"] = df["text"]
def remove_stopwords(text):
  words = word_tokenize(text)
  filtered_words = [word for word in words if word.lower() not in stop_words]
  return "".join(filtered_words)
  df["text"] = df["text"].apply(remove_stopwords)
  print("Before and After Removing Stopwords:\n")
  for i in range(5):
    print(f"Sample {i+1}")
    print("Before :", df["before_stopwords"].iloc[i])
    print("After :", df["text"].iloc[i])
    print("-" * 60)

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

8. (Complete Text Cleaning Pipeline)
   Code:

```
[19]
✓ 0s
```

```python
import re
import string
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')

stop_words = set(stopwords.words('english'))

def clean_text(text):
  text = text.lower()

  text = text.translate(str.maketrans('', '', string.punctuation))

  text = re.sub(r'\d+', '', text)

  text = text.encode('ascii', 'ignore').decode('ascii')

  words = word_tokenize(text)
  words = [word for word in words if word not in stop_words]

  return "".join(words)

  df["cleaned_text"] = df["original_text"].apply(clean_text)

  print(df[["original_text", "cleaned_text"]].head(10))
```

Output:

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
```

9. (Text Length Analysis)

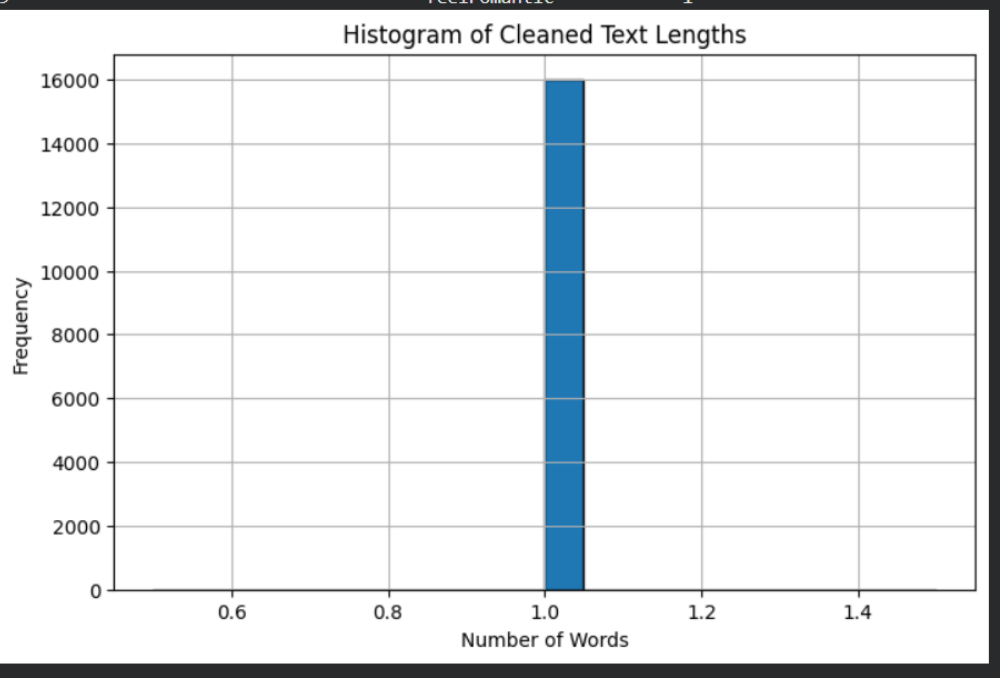
Code:

```python
import matplotlib.pyplot as plt

df["text_length"] = df["cleaned_text"].apply(lambda x: len(str(x).split()))
print("Text Lengths:")
print(df[["cleaned_text", "text_length"]].head(10))
plt.figure(figsize=(8,5))
plt.hist(df["text_length"], bins=20, edgecolor="black")
plt.title("Histogram of Cleaned Text Lengths")
plt.xlabel("Number of Words")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()

print("\nText Length Statistics")
print("-------------------------")
print("Average Text Length:", round(df["text_length"].mean(), 2))
print("Minimum Text Length:", df["text_length"].min())
print("Maximum Text Length:", df["text_length"].max())
```

Output:

```
Text Lengths:
                                        cleaned_text  text_length
0                                   didntfeelhumiliated            1
1  gofeelinghopelessdamnedhopefularoundsomeonecar...            1
2                imgrabbingminutepostfeelgreedywrong            1
3     everfeelingnostalgicfireplaceknowstillproperty            1
4                                     feelinggrouchy            1
5            ivefeelinglittleburdenedlatelywasntsure            1
6  ivetakingmilligramstimesrecommendedamountivefa...            1
7            feelconfusedlifeteenagerjadedyearoldman            1
8  petronasyearsfeelpetronasperformedwellmadehuge...            1
9                                     feelromantic            1
```

9 feelromantic 1




```
Text Length Statistics
-------------------------
Average Text Length: 1.0
Minimum Text Length: 1
Maximum Text Length: 1
```

10. (Mini Project – Full Preprocessing Pipeline)

Code:

```
#====================================
# 1. Load the dataset
# ====================================

df = pd.read_csv(
    "/content/train.txt",
    sep=";",
    names=["text", "emotions"]
)

print("Dataset Shape:", df.shape)

# ================================
# 2. Encode the emotion labels
# ================================

label_encoder = LabelEncoder()
df["emotion_encoded"] = label_encoder.fit_transform(df["emotions"])
print("\nEmotion Label Mapping:")
print(dict(zip(label_encoder.classes_, label_encoder.transform(label_encoder.classes_))))
```

```
# ================================
# 3. Complete Text Cleaning
# ================================

nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)
nltk.download("stopwords", quiet=True)

stop_words = set(stopwords.words("english"))

def clean_text(text):

  text = str(text).lower()

  text = text.translate(
      str.maketrans("", "", string.punctuation)
  )

  text = re.sub(r"\d+", "", text)

  text = text.encode("ascii", "ignore").decode("ascii")

  words = word_tokenize(text)
  words = [
      word for word in words
      if word not in stop_words
  ]

  return "".join(words)
```

```
    return "".join(words)

df["cleaned_text"] = df["text"].apply(clean_text)

# ====================================
# 4. Save cleaned dataset
# ====================================

df.to_csv(
    "/content/cleaned_emotions.csv",
    index=False
)

print("\nCeaned dataset saved as:")
print("/content/cleaned_emotions.csv")

# ================================
# 5. Show value counts of each emotion
# ====================================

print("\nEmotion Value Counts:")
print(df["emotions"].value_counts())
```

```
print(df["emotions"].value_counts())

# ========================================
# Display sample
# ========================================

print("\nFirst 10 rows:")
print(df[["text", "cleaned_text", "emotions", "emotion_encoded"]].head(10))
```

Output:

```
Dataset Shape: (16000, 2)

Emotion Label Mapping:
{'anger': np.int64(0), 'fear': np.int64(1), 'joy': np.int64(2), 'love': np.int64(3), 'sadness': np.int64(4), 'surprise': np.int64(5)}

Ceaned dataset saved as:
/content/cleaned_emotions.csv

Emotion Value Counts:
emotions
joy         5362
sadness     4666
anger       2159
fear        1937
love        1304
surprise     572
Name: count, dtype: int64
```

```
First 10 rows:
                                                text  \
0                            i didnt feel humiliated
1  i can go from feeling so hopeless to so damned...
2   im grabbing a minute to post i feel greedy wrong
3  i am ever feeling nostalgic about the fireplac...
4                              i am feeling grouchy
5  ive been feeling a little burdened lately wasn...
6  ive been taking or milligrams or times recomme...
7  i feel as confused about life as a teenager or...
8  i have been with petronas for years i feel tha...
9                                i feel romantic too

                                      cleaned_text  emotions  \
0                                 didntfeelhumiliated   sadness
1  gofeelinghopelessdamnedhopefularoundsomeonecar...   sadness
2                imgrabbingminutepostfeelgreedywrong     anger
3     everfeelingnostalgicfireplaceknowstillproperty      love
4                                    feelinggrouchy     anger
5            ivefeelinglittleburdenedlatelywasntsure   sadness
6  ivetakingmilligramstimesrecommendedamountivefa...  surprise
7            feelconfusedlifeteenagerjadedyearoldman      fear
8  petronasyearsfeelpetronasperformedwellmadehuge...       joy
9                                    feelromantic      love

   emotion_encoded
0                4
1                4
2                0
3                3
4                0
5                4
6                5
7                1
8                2
9                3
```